Q1:

a.

In Scale_down function, we apply a Gaussian blur to the image to suppress high-frequency noise, which can cause aliasing when resizing.

```
blurred = cv2.GaussianBlur(image, ksize: (5, 5), sigmaX: 5)
```

Then the function converts the image to the frequency domain using FFT and shifts the zero-frequency component to the center for easier manipulation.

```
f_transform = np.fft.fft2(blurred)
f_shift = np.fft.fftshift(f_transform)
```

Afterwards, we want to remove high-frequency components to keep the low frequency components so we crop the central region of the frequency spectrum.

```
# Compute cropping indices
h_start, h_end = (h - new_h) // 2, (h + new_h) // 2
w_start, w_end = (w - new_w) // 2, (w + new_w) // 2

# Crop the frequency spectrum
f_cropped = f_shift[h_start:h_end, w_start:w_end]
```

Finally, return the image back to the spatial domain using inverse fourier and normalize it.

```
# Step 4: Perform Inverse FFT to obtain the downscaled image
f_ishift = np.fft.ifftshift(f_cropped)
img_scaled = np.abs(np.fft.ifft2(f_ishift))

# Normalize and return image
img_scaled = np.uint8(cv2.normalize(img_scaled, dst: None, alpha: 0, beta: 255, cv2.NORM_MINMAX))

return img_scaled
```

b. We start by converting the image to the frequency domain using FFT then shipting the zero-frequency component to the center.

```
# Step 1: Compute 2D Fourier Transform
f_transform = np.fft.fft2(image)
f_shift = np.fft.fftshift(f_transform)
```

Then we create a zero matix of the new size of the image so then we can use it to zero pad the image.

```
# Step 2: Zero-Padding in Frequency Domain
h, w = f_shift.shape
new_h, new_w = int(h * resize_ratio), int(w * resize_ratio)

# Create a zero-padded frequency matrix
f_padded = np.zeros( shape: (new_h, new_w), dtype=f_shift.dtype)
```

Then we insert the image in the middle of the zero matrix, ensuring that the low-frequency components remain intact, while high-frequency details are smoothly interpolated.

```
# Compute insertion indices
h_start, h_end = (new_h - h) // 2, (new_h + h) // 2
w_start, w_end = (new_w - w) // 2, (new_w + w) // 2

# Insert original frequency content into the center of the new matrix
f_padded[h_start:h_end, w_start:w_end] = f_shift
```

Finally, return the image back to the spatial domain using inverse fourier and normalize it.

```
# Step 3: Perform Inverse FFT to obtain the upscaled image
f_ishift = np.fft.ifftshift(f_padded)
img_scaled = np.abs(np.fft.ifft2(f_ishift))

# Normalize image and return
img_scaled = np.uint8(cv2.normalize(img_scaled, dst: None, alpha: 0, beta: 255, cv2.NORM_MINMAX))
return img_scaled
```

c. Here, we want to calculate the NCC between the image and the patter. Therefore, we first smooth the image using gaussian blur to reduce noise.

```
image = cv2.GaussianBlur(image, ksize: (5, 5), sigmaX: 1)
```

Then, divide the image into windows of size the patter where the windows are overlapping, creating every possible window of the pattern size.

```
# Extract windows from the image with the same shape as the pattern
windows = sliding_window_view(image, pattern.shape)  # Shape: (H', W', pattern_H, pattern_W)
```

Now we calculate the mean for the windows and the pattern.

```
# Compute mean and standard deviation for each image window
window_means = np.mean(windows, axis=(-2, -1), keepdims=True)
# Compute mean and standard deviation for the pattern
pattern_mean = np.mean(pattern)
```

now we calculate the NCC using the formula:

$$\frac{\sum_{x,y \in N} \left[I(u+x, v+y) - \bar{I}_{uv}\right]\left[P(x,y) - \bar{P}\right]}{\left[\sum_{x,y \in N} \left[I(u+x, v+y) - \bar{I}_{uv}\right]^2 \sum_{x,y \in N} \left[P(x,y) - \bar{P}\right]^2\right]^{1/2}}$$

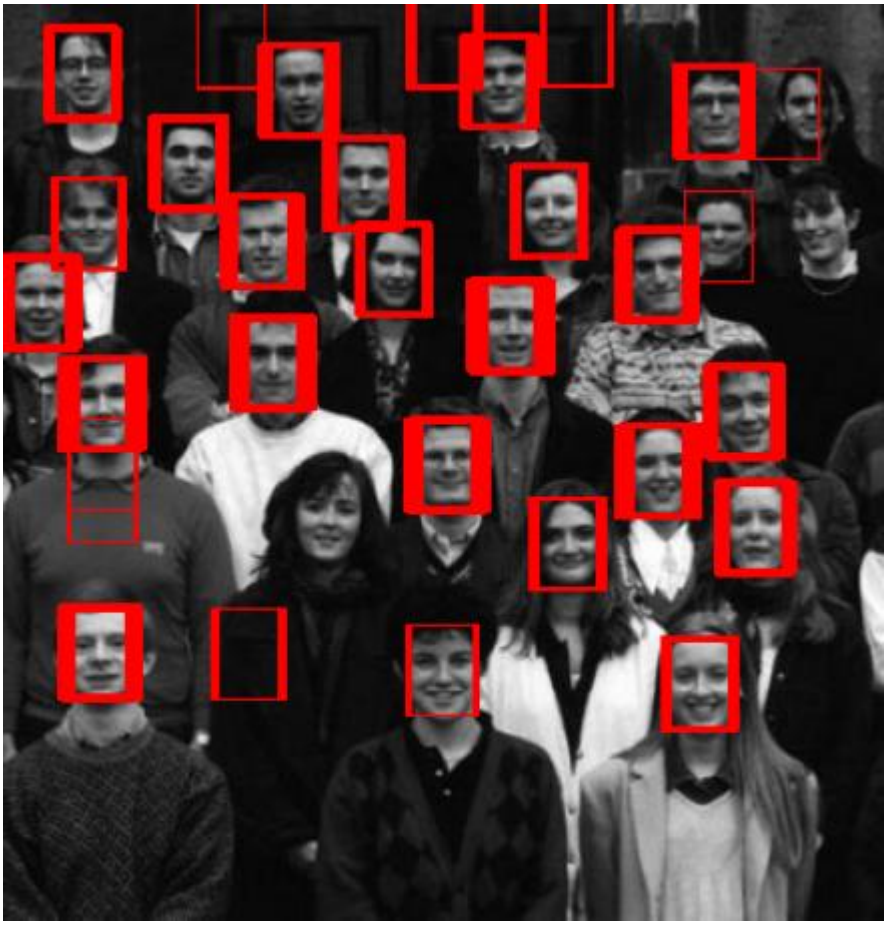
```
# Compute NCC (avoid division by zero)
numerator = np.sum((windows - window_means) * (pattern - pattern_mean), axis=(-2, -1))
denominator = np.sqrt(np.sum((windows - window_means) ** 2, axis=(-2, -1)) * np.sum((pattern - pattern_mean) ** 2))
```

Set the NCC to 0 where the denominator is 0 (since we can't know any information).

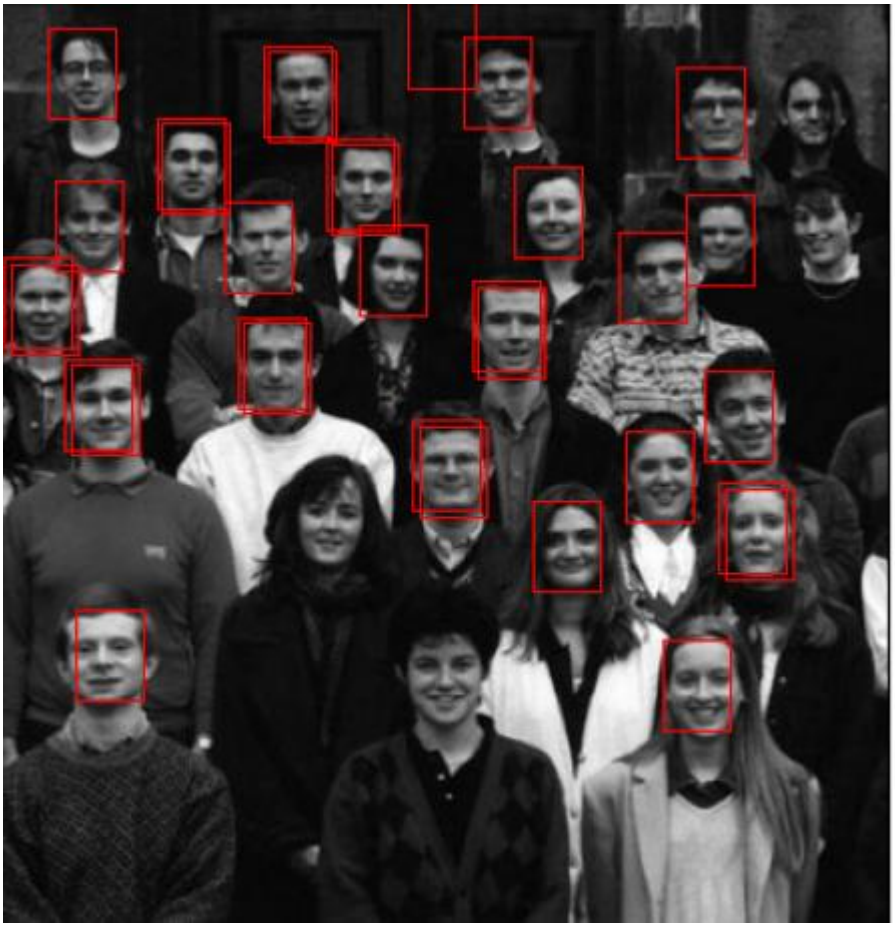
```
denominator[denominator == 0] = 1
numerator[denominator == 0] = 0
ncc = numerator / denominator
```

The results we get are:
For the students image:
This is the result that we got originally, with the following parameters:

```
scale_up_ratio = 1.6
scale_down_ratio = 0.79
threshold = 0.535
```



But after some further modifications(using non maximum suppression with window size $3 \times 3$, and threshold = 0.49), we managed to get this result:



For the crew image:



We got this result by setting the following variables:

```
scale_up_ratio = 2.52
scale_down_ratio = 0.58
threshold = 0.51
```

Implementing the main was straightforward; we read the images, convert them to grayscale, scaled them, then calculated NCC, got the points greater than threshold, resized them their original size in the image.

Q2:

a. In this part we have implemented a function to build a Laplacian pyramid, which breaks an image into several levels of detail. Each level is a smaller, more blurred version of the previous one, and we calculate the lost details then between levels. These lost details are saved as Laplacian layers, while the smallest and most blurred image is at the top of the pyramid. This process helps us represent the image at multiple scales, making it easier to blend two images smoothly after.

```python
def get_laplacian_pyramid(image, levels, resize_ratio=0.5):  3 usages
    pyramid = []
    img = np.float32(image)  # the image is float to avoid math issues later

    for level in range(levels - 1):
        # Applying gaussian blur to the given image
        blurred_img = cv2.GaussianBlur(img, ksize: (5, 5), sigmaX: 0)
        # Down sampling the image by 2 - half of it's size
        smaller_img = cv2.resize(blurred_img, dsize: (0, 0), fx=resize_ratio, fy=resize_ratio)
        # Up sampling the image to its original size
        bigger_img = cv2.resize(smaller_img, dsize: (img.shape[1], img.shape[0]))

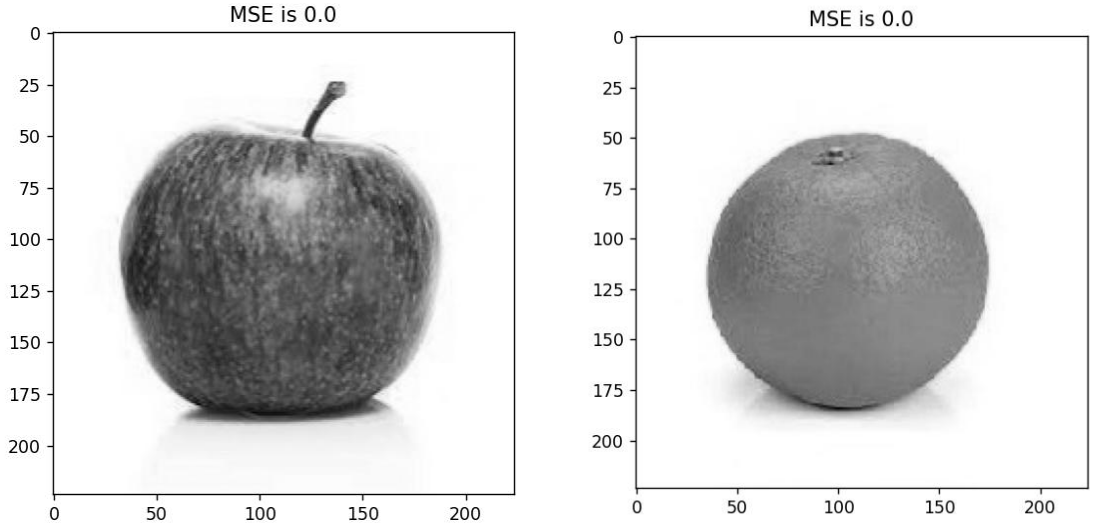
        laplacian = img - bigger_img  # finding the details in the laplacian layer
        pyramid.append(laplacian)  # adding the laplacian layer to the pyramid
        img = smaller_img  # moving on to the next level to compute it

    # adding the smallest image to the top of the pyramid
    pyramid.append(img)
    # and returning it
    return pyramid
```

b. In this section, we have implemented a function called restore_from_pyramid that takes a Laplacian pyramid and reconstructs the original image by collapsing it.
This involves sampling and adding layers back together, starting from the smallest image and working up to the full size.

```python
def restore_from_pyramid(pyramidList, resize_ratio=2):  2 usages
    updated_image = pyramidList[-1]  # Starting with the smallest image which is the blurriest on the top of the pyramid
    # looping from top to bottom(in the pyramid levels)
    for l in range(len(pyramidList) - 2, -1, -1):
        # resize so it can be added to the next layer in the pyramid
        updated_image = cv2.resize(updated_image, dsize: (0, 0), fx=resize_ratio, fy=resize_ratio)
        # adding it
        updated_image = cv2.add(pyramidList[l], updated_image)
    # returning the fully restored image - all of it
    return updated_image
```

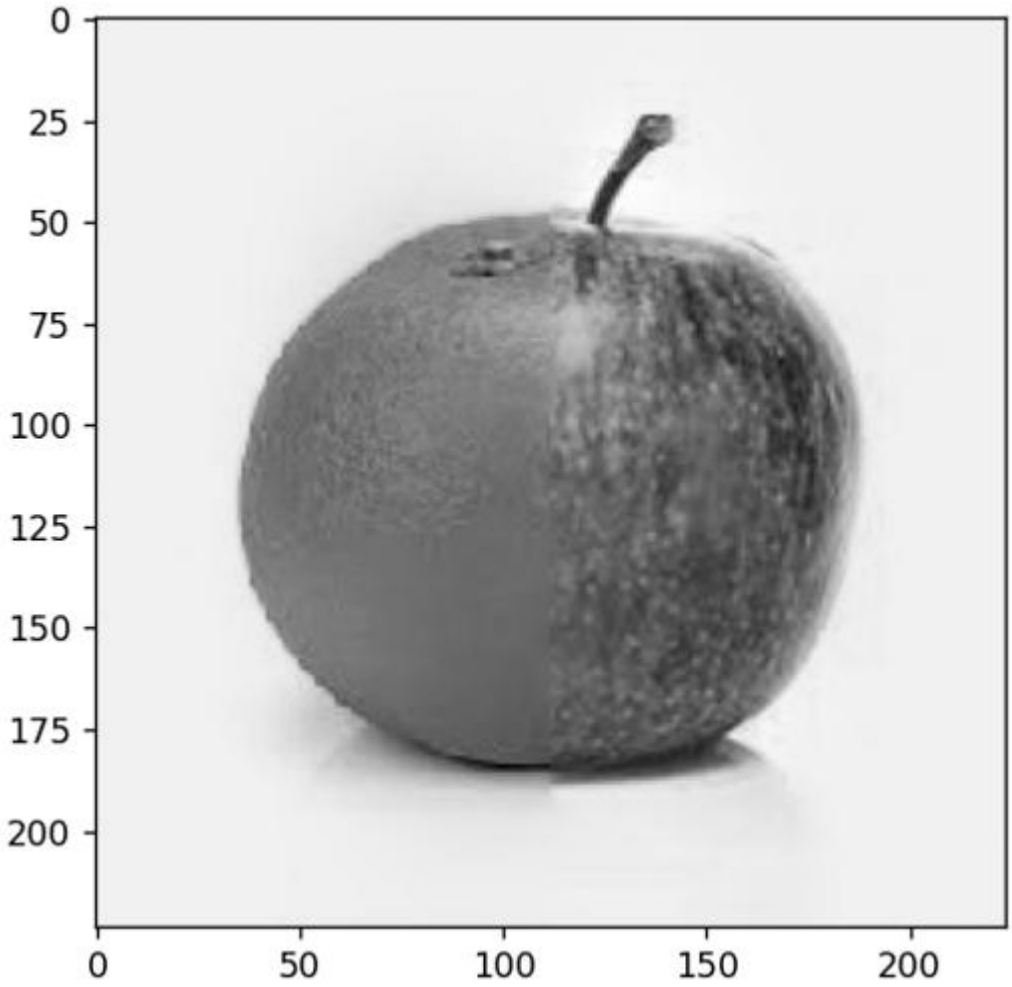
After implementing this process, we have used validate_operation to check our reconstruction's accuracy. The target was to achieve an MSE which is lower than 1, meaning the reconstructed image is almost identical to the original one.




c.

In this blending step, we have combined the Laplacian pyramids of the apple and orange across 5 levels. Each level has a mask that decides how much of each image to keep. The mask starts with 1.0 (fully apple) on the left side, until we reach 0.5 * width - level, where the blending starts. In the middle blending zone, which is 2 * level columns wide, the mask gradually decreases from 0.9 to 0.0. So, this gradual cross dissolve smoothly mixes the apple and orange. At the top levels, which are the blurry images, the blending zone is wider, while at the lower levels, which have sharper details, the blend is tighter. At the end, after blending all levels this way, the final blended pyramid collapsed back into a single image.

Here’s the final result:




And this is the code:

```python
def blend_pyramids(levels):  1 usage
    blended_pyramid = []

    for l in range(levels):
        #get each current level of the images in both pyramids
        apple_layer = pyr_apple[l]
        orange_layer = pyr_orange[l]

        #getting the size of this current level
        rows, cols = apple_layer.shape

        # Create the mask in the same size as the image
        mask = np.zeros( shape: (rows, cols), dtype=np.float32)

        mid = cols // 2 #finding the middle of an image
        start_blend = int(0.5 * cols - l)
        end_blend = int(0.5 * cols + l)

        #starting from the left side
        for c in range(start_blend):
            mask[:, c] = 1.0 #Apple side(full size)

        #The blending zone
        blend_size = end_blend - start_blend  # The width of the blending area

        #looping column by column in the blended area that we are working on
        for b in range(blend_size):
            col = start_blend + b  # The exact column we are working on currently
            mask[:, col] = 0.9 - (0.9 * b / (2 * l))

        #Blending both images using the mask
        blended_layer = orange_layer * mask + apple_layer * (1 - mask)

        #Adding this blended layer to the result pyramid
        blended_pyramid.append(blended_layer)

    return blended_pyramid
```

Improvement- try.py file:
The main difference here is that in this code we used a sharp mask (half white and half black) to perform the smooth transition between the images.

```python
# Create a Sharper Mask Instead of a Gradient
mask = np.zeros( shape: (rows, cols), dtype=np.uint8)
mask[:, cols//2:] = 255  # Apple on the right, Orange left
```

Afterwards, we created a function to generate the gaussian pyramid and the Laplacian. Where the gaussian pyramid is done by the built-in function cv2.pryDown which blurs the image at each level by downsampling.
The Laplacian pyramid is made by expanding the current level to the size of the previous level then subtracting them. Which creates a high frequency level (which is the Laplacian).

```python
# Function to Generate Gaussian & Laplacian Pyramids
2 usages
def generate_pyramids(image, levels=5):  # Reduced to 5 levels for better detail
    gp = [image]
    for _ in range(levels):
        image = cv2.pyrDown(image)
        gp.append(image)

    lp = [gp[-1]]  # Smallest Gaussian image
    for i in range(levels, 0, -1):
        expanded = cv2.pyrUp(gp[i], dstsize=(gp[i-1].shape[1], gp[i-1].shape[0]))
        laplacian = cv2.subtract(gp[i-1], expanded)
        lp.append(laplacian)
    return gp, lp
```

So, we generated the pyramids for the 2 images.

```python
# Generate pyramids
gp1, lp1 = generate_pyramids(img1)
gp2, lp2 = generate_pyramids(img2)
```

Afterwards, we created the gaussian pyramid of the mask making sure to have the same number of levels as the Laplacian pyramid of the images.

```python
# Ensure Gaussian Mask Pyramid Matches Laplacian Pyramid
gp_mask = [mask]
for _ in range(len(lp1) - 1):
    mask = cv2.pyrDown(mask)
    gp_mask.append(mask)
```

Now, given the Laplacian pyramids and the gaussian pyramid of the mask, we can start the blending process.
We blend the Laplacian pyramids level by level using the gaussian pyramid of the mask.

```python
# Blend Laplacian Pyramids Using the Mask Pyramid
lp_blend = []
for l1, l2, gm in zip(lp1, lp2, gp_mask):
    gm_resized = cv2.resize(gm, dsize: (l1.shape[1], l1.shape[0]))  # Ensure size matches
    gm_resized = gm_resized[:, :, np.newaxis]  # Reshape for broadcasting

    l_blend = l1 * gm_resized + l2 * (1 - gm_resized)
    lp_blend.append(l_blend)
```

Finally, reconstruct the blended image from the Laplacian pyramid by upsampling the current level and adding it to the next level.

```python
# Reconstruct Final Blended Image
blended = lp_blend[0]
for i in range(1, len(lp_blend)):
    blended = cv2.pyrUp(blended, dstsize=(lp_blend[i].shape[1], lp_blend[i].shape[0]))
    blended = cv2.add(blended, lp_blend[i])
```

And we got this result at the end:



I think we can do better using CNN.